

Transformers and reinforcement learning for the team orienteering problem under dynamic conditions

Guerrero, Escoto, Ammouriova, Men and Juan — Universitat Politècnica de València

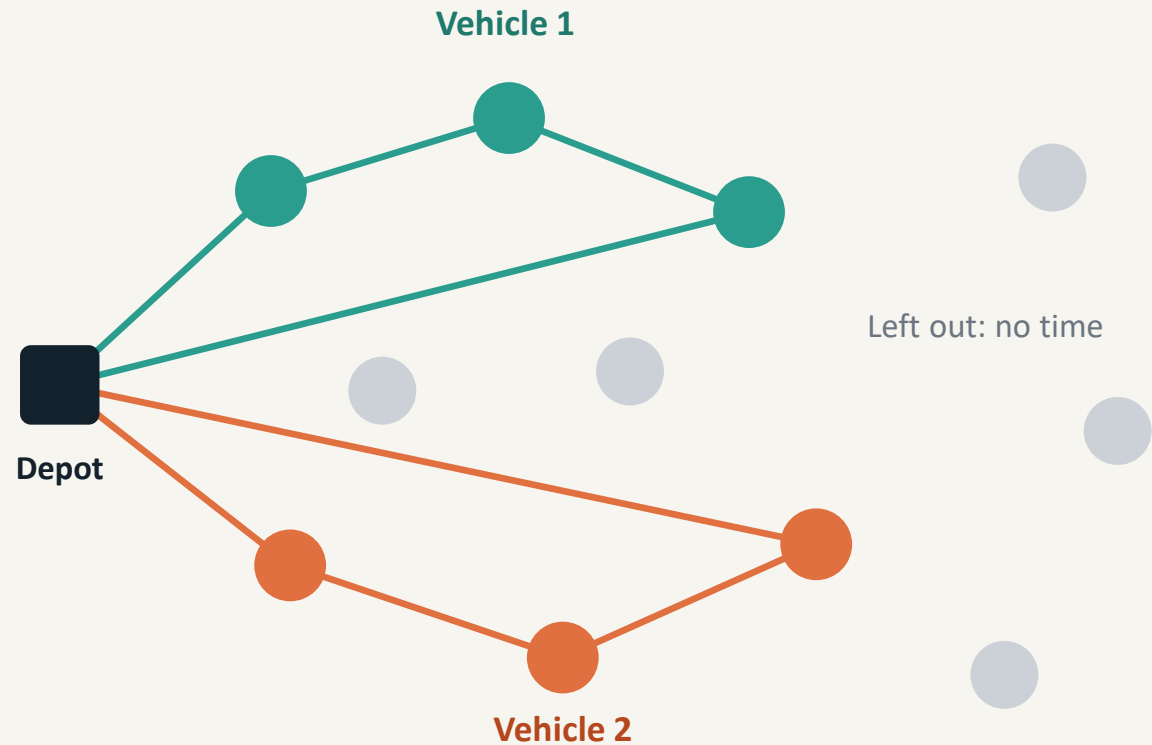
The team orienteering problem

Every stop carries a reward.

All vehicles leave from one depot and must reach the end depot.

Each vehicle has a time budget, so some stops stay unvisited.

Maximise reward, not coverage.



Why it matters: last-mile delivery with electric vehicles, where battery range acts as the time budget.

NP-hard: the number of possible route sets explodes with the number of stops.

Travel time is not a constant

$$f(i, j) = d(i, j) \times (1 + 0.0625 \cdot \text{traffic} + 0.0625 \cdot \text{weather})$$

$d(i, j)$

The plain distance on the map.

traffic

Congestion, from 0 (clear) to 1 (jammed).

weather

Rain, wind or snow, also 0 to 1.

Planned leg



Worst-case leg



+12.5%

Worst case, a leg takes 12.5% longer than the plan assumed.

A plan that fits exactly in the budget is therefore a plan that will break

Both methods loop. They loop at different moments.

Search — the VNS baseline



The loop runs while the courier waits: VNS found its best solution after **4.55 s** on average (limit: 20 s).

Learning — the RL model



The loop runs once, offline: about **40 hours** for the dynamic models. Afterwards a plan takes **0.34 s** (deterministic) to 0.54 s (dynamic, best variant).

Read the map once, then decide stop by stop

ONCE
per instance

The instance: stops, rewards,
vehicle budgets

Embed: each stop and vehicle
becomes a vector

Encode: attention relates every
stop to the rest

EVERY
step of the route

Refresh the state: masked
attention hides visited stops
and finished vehicles

Build the context: where am I,
how much time is left, what did
I do before

Which stops can I reach and still
get home? Pick one and move

until every vehicle is back at the end depot

The embeddings are fixed per instance. At each step only the visited-stop mask, the context and the list of allowed stops change.

Encoder–decoder transformer, as in Vaswani et al. and Kool et al.

The idea that makes it work

A STOP IS OFFERED ONLY IF

cost to reach it + cost to get home from it $\leq$ time left

The way there is predicted

A small network (an MLP) estimates what traffic and weather will cost on that exact segment.

The way home is guessed high

Conditions later on cannot be known, so the plain distance is inflated by a learned worst-case factor.

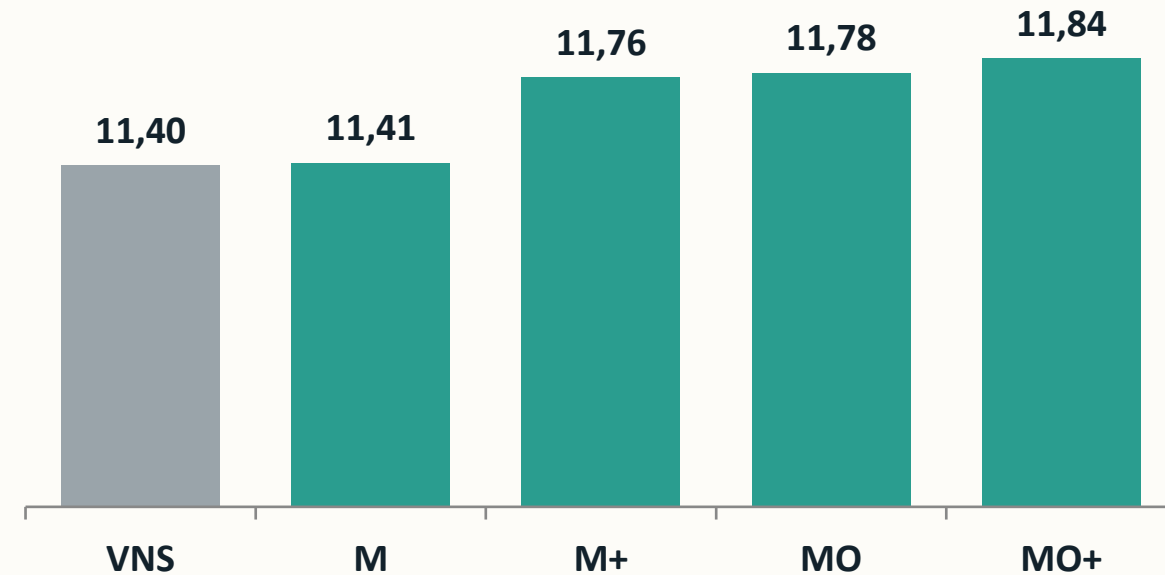
One loss trains both: REINFORCE for the routing, mean squared error for the prediction, joined by a weight λ .

$$L = -(R(\pi) - b(s)) \cdot \log p_{\theta}(\pi \mid s) + \lambda \cdot \text{MSE}$$

Feasibility is enforced by construction, not repaired afterwards

Quality versus time, by variant

AVERAGE REWARD, HIGHER IS BETTER



Axis starts at 10

SECONDS TO PRODUCE ONE PLAN

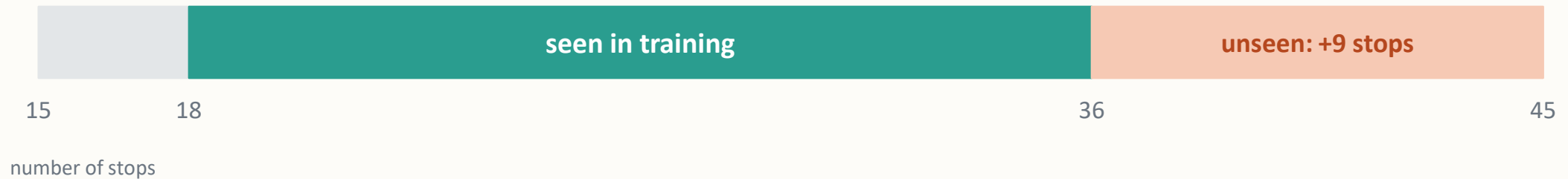


MO+ is about 4% better than VNS and 13× faster. M matches VNS in over 3,000× less time.

M: base model · M+: with reflections · MO: with rotations · MO+: both (256 versions of each instance). Deterministic problem, 15–45 stops, 2–4 vehicles.

Generalisation, and where each method wins

ONE MODEL, TRAINED ON 18–36 STOPS AND 2–4 VEHICLES, TESTED UP TO 45 STOPS WITHOUT RETRAINING



REWARD ADVANTAGE OF RL (MO+) OVER VNS

+0.18

2 vehicles

+0.47

3 vehicles

+0.69

4 vehicles

VNS catches up as stops grow

It gets its best result at 45 stops and 2 vehicles.

More time barely helps VNS

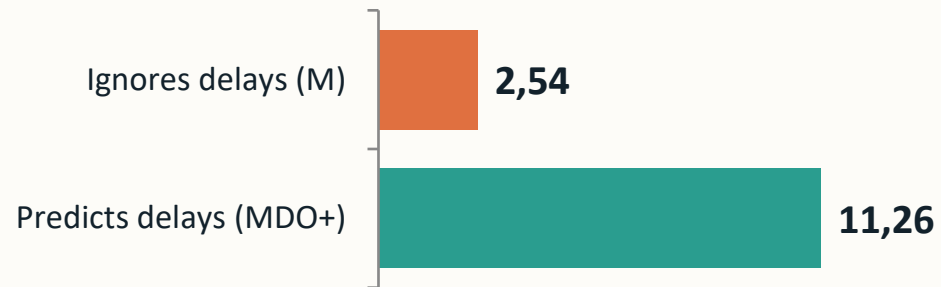
Giving VNS 3 min instead of 20 s improved it by only ~0.2% (35 stops).

Average reward on the deterministic instances; VNS limited to 20 s per instance

When travel times change

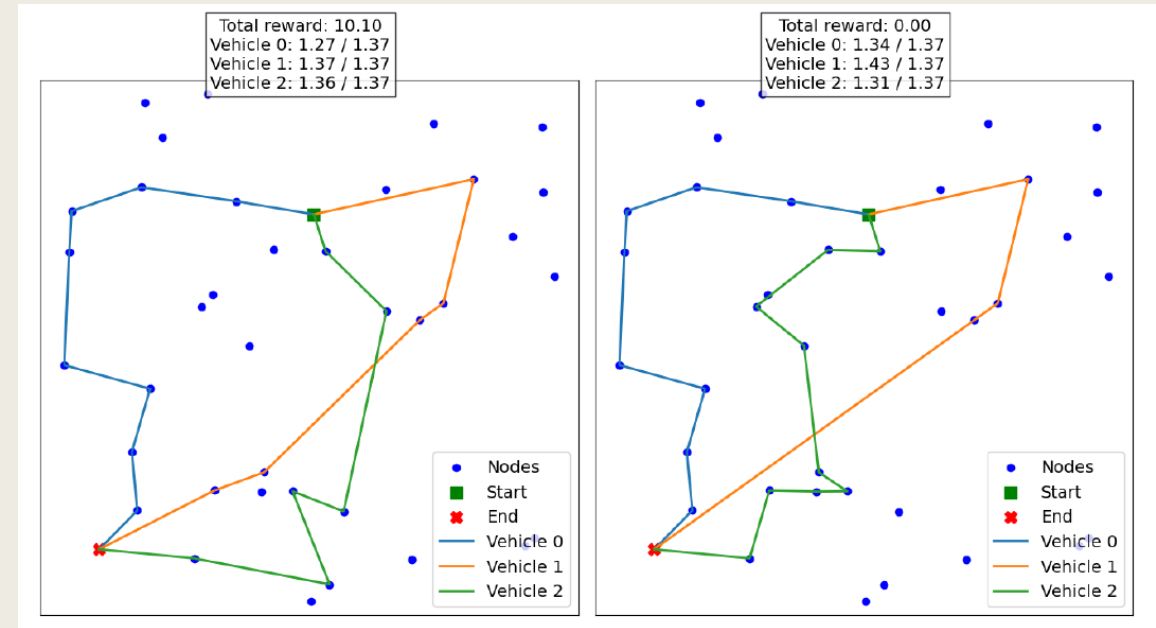
AVERAGE REWARD, DYNAMIC SCENARIO

No prediction module vs. best dynamic model



2.54 is not bad routing. A vehicle that arrives late has its route thrown out, and it scores nothing.

The dynamic models broke the limit zero times.



Same model, same instance. Left: no delays, reward 10.10. Right: with delays, vehicle 1 needs 1.43 of a 1.37 budget, so the reward drops to 0.00.

Figure 5 from Guerrero et al. (2025), CC BY 4.0

TAKEAWAY · OUR READING OF THE RESULTS

Modelling the uncertainty mattered more than solving the problem better.

WORTH ASKING ABOUT

Training and test instances come from the same uniform distribution.

About 40 hours of training for each dynamic model, and more resources as instances grow.

No re-optimisation once a vehicle is on the road, and no stochastic travel times yet.

NEXT STEPS IN THE PAPER

Stochastic conditions: probabilistic travel times and fluctuating customer availability.

A hybrid with metaheuristics such as VNS, combining global search with RL's speed.

Thank you — questions?

Mathematical formulation

Objective	$\max \sum_v \sum_{i,j} x_{ijv} \cdot r_j$	collect as much reward as possible
Depots	$\sum_j x_{ojv} \leq 1 \quad \cdot \quad \text{no arc enters } o \text{ or leaves } d$	each vehicle leaves o at most once and ends at d
Visits	$\sum_v \sum_i x_{ijv} \leq 1$	each stop is visited at most once across all vehicles
Routes	flow conservation + MTZ ordering variables	continuous routes without subtours
Time budget	$\sum_{i,j} x_{ijv} \cdot f(i, j) \leq L_v$	each vehicle stays within its limit

$f(i, j) = d_{ij} \cdot (1 + \alpha \cdot t_{ij} + \beta \cdot e_{ij})$, predicted before optimising. Costs are assumed known or estimated at optimisation time; no re-optimisation during execution.

How the instances are generated

$[0, 1]^2$

Nodes placed uniformly at random in the unit square

$U[0, 1]$

Reward of each node, drawn uniformly at random

$d(o, d) + 0.5 + U[0, 1.5]$

Distance budget of each vehicle: feasible, but never trivial

$15-45 \cdot 2-4$

Stops · vehicles. 600 instances per combination (300 deterministic and their dynamic versions)

Dynamic version: the true travel time is the distance inflated by up to 12.5%, with traffic and weather each between 0 (ideal) and 1 (most adverse).

Training and decoding details

Training

- REINFORCE with a baseline built from augmented instances (Lee and Ahn)
- 2,700 optimisation steps per epoch, batch of 256
- Each epoch is evaluated on 110,000 instances; a new best model is kept only if a t-test (0.05) confirms it
- One model for 18–36 stops and 2–4 vehicles, each configuration sampled equally
- Adam, learning rate decayed, then sampled around the best rate
- RTX 4060 GPU, 16 GB of RAM; about 40 h per dynamic model

Decoding at inference

- M: greedy decoding, the most probable stop at each step
- M+: also solves 8 reflections of the instance and keeps the best
- MO: also solves 32 rotations of the instance and keeps the best
- MO+: reflections $\times$ rotations = 256 versions of each instance

Mean time per plan. Deterministic: 0.0014 s, 0.044 s, 0.07 s, 0.34 s. Dynamic: 0.0022 s, 0.052 s, 0.095 s, 0.54 s.